

MODELO 1

QUESTÃO 01 – VALOR 0,25 PONTO

Cenário:

Um pipeline de limpeza de texto de um e-commerce recebe a seguinte lista de palavras extraída de um chamado de cliente:

tokens = ["os", "produtos", "comprados", "não", "chegaram"]

Você precisa filtrar as palavras de parada (*stopwords*) de alta frequência e aplicar a lematização manual do texto.

1. Crie o trecho de um código em Python que receba a lista tokens, remova as stopwords "os" e "não", e substitua as palavras "comprados" e "chegaram" pelas suas formas de dicionário (lemas): "comprar" e "chegar". Imprima a lista final.
2. Qual a diferença entre **Stemming** e **Lematização**? Por que a Lematização garante que a palavra resultante exista no dicionário e o Stemming não?

a) Resposta:

tokens = ["os", "produtos", "comprados", "não", "chegaram"]

Remoção de stopwords e substituição manual por lemas

stopwords = ["os", "não"]

lemas_map = {"comprados": "comprar", "chegaram": "chegar"}

tokens_finais = [lemas_map.get(w, w) for w in tokens if w not in stopwords]

print(tokens_finais)

Saída esperada: ['produtos', 'comprar', 'chegar']

1º parâmetro (w): Procura se a palavra atual existe como chave no dicionário lemas_map.

2º parâmetro (w): É o valor de fallback (padrão). Se a palavra NÃO for encontrada no dicionário, o .get() devolve a própria palavra sem alterá-la!

b) Resposta Esperada:

- O Stemming corta o sufixo da palavra de forma arbitrária (por regras de prefixos e sufixos), gerando apenas um radical (ex: "compr"). A Lematização utiliza análise gramatical e um dicionário/léxico para reduzir a palavra à sua forma canônica/dicionário (ex: "comprar").
- Por que garante no dicionário? A Lematização faz uma análise morfológica completa consultando vocabulários oficiais da língua, enquanto o Stemming aplica apenas regras heurísticas brutas de corte no final do texto.

QUESTÃO 02 – VALOR 0,25 PONTO

Cenário:

Um chatbot bancário recebe uma mensagem e seu modelo de NLU calcula uma confiança de 55% (0.55) para a intenção "realizar_pix". A regra de negócio do banco exige um Limiar de Confiança (Threshold) mínimo de 70% (0.70) para autorizar qualquer transação.

1. Escreva uma função em Python chamada `validar_intencao(confianca)` que receba o valor numérico da confiança. Se for maior ou igual a 0.70, retorne "Ação Executada". Caso contrário, retorne "Fallback Acionado". Teste a função com o valor 0.55.
2. Qual é a função do **Threshold (Limiar)** e do **Fallback** em um chatbot? O que aconteceria de errado no sistema se o limiar fosse configurado em apenas 20% (0.20)?

a) Resposta:

```
def validar_intencao(confianca):  
    if confianca >= 0.70:  
        return "Ação Executada"  
    else:  
        return "Fallback Acionado"  
  
# Teste solicitado  
print(validar_intencao(0.55))  
  
# Saída esperada: "Fallback Acionado"
```

b) Resposta Esperada:

- **Função:** O **Threshold (Limiar)** define a régua mínima de segurança/certeza que a IA precisa ter para agir. O **Fallback** é o mecanismo de contingência ativado quando o threshold não é atingido (evita executar uma ação sem certeza).
- **Limiar muito baixo (ex: 0.10):** O sistema aceitaria qualquer palpite fraco da IA, resultando em respostas totalmente erradas, transações indevidas na conta do cliente e falhas graves de experiência/segurança.

MODELO 2

QUESTÃO 01 – VALOR 0,25 PONTO

Uma imobiliária utiliza o algoritmo TF-IDF para classificar chamados recebidos. As palavras "apartamento" e "vazamento" foram analisadas pelo algoritmo em uma base de dados onde a palavra "apartamento" aparece em todas as mensagens e "vazamento" aparece apenas em chamados raros de urgência.

1. Crie o trecho onde uma função chamada *calcular_peso_tfidf(frequencia_geral)* que receba a frequência de aparecimento da palavra no sistema. Se a palavra for "muito_frequente", retorne o peso 0.1. Se for "pouco_frequente", retorne o peso 0.9. Teste para a palavra "vazamento" (pouco frequente).
2. Qual a lógica conceitual do TF-IDF para atribuir peso baixo a palavras muito comuns e peso alto a palavras raras?

a) Resposta:

```
def calcular_peso_tfidf(frequencia_geral):
```

```
    if frequencia_geral == "muito_frequente":
```

```
        return 0.1
```

```
    elif frequencia_geral == "pouco_frequente":
```

```
        return 0.9
```

```
# Teste para "vazamento"
```

```
print(calcular_peso_tfidf("pouco_frequente"))
```

```
# Saída esperada: 0.9
```

b) Resposta Esperada:

- **Lógica do TF-IDF:** Ele multiplica a frequência do termo no documento (TF) pelo inverso da frequência no corpus (IDF). Palavras muito comuns (como "apartamento" ou pontuações) aparecem em quase todos os textos e possuem baixo valor informativo. Palavras raras (como "vazamento") trazem o verdadeiro contexto específico/urgente do documento, recebendo assim um peso estatístico maior.

QUESTÃO 02 – VALOR 0,25 PONTO

Um modelo de IA de e-commerce foi testado para identificar **Transações Fraudulentas**. Em um teste com compras reais, o modelo gerou os seguintes resultados:

- Verdadeiros Positivos (VP) = 80
 - Falsos Positivos (FP) = 20
 - Falsos Negativos (FN) = 5
1. Escreva o trecho de um código em Python que calcule e exiba a **Precisão** do modelo utilizando a fórmula: $Precisão = \frac{VP}{VP+FP}$.
 2. O que representa um erro de **Falso Positivo** e um erro de **Falso Negativo** nesse cenário de fraude bancária? Qual dos dois causa prejuízo financeiro direto?

a) Resposta:

VP = 80

FP = 20

FN = 10

```
# Precisão = VP / (VP + FP)
```

```
precisao = VP / (VP + FP)
```

```
print(f'Precisão: {precisao:.2f} ({precisao * 100:.1f}%)')
```

```
# Saída esperada: 0.80 (80.0%)
```

b) Resposta Esperada:

- **Falso Positivo (FP):** O sistema aponta que uma transação legítima é fraude (bloqueia o cartão do cliente de forma indevida).
- **Falso Negativo (FN):** O sistema deixa passar uma transação verdadeiramente golpista/fraudulenta sem bloquear.
- **Prejuízo direto:** O **Falso Negativo** causa prejuízo financeiro direto ao banco/loja, pois a fraude é consumada e o valor é perdido.

MODELO 3

QUESTÃO 01 – VALOR 0,25 PONTO

Um cliente digita a seguinte mensagem no assistente virtual de uma farmácia: "Quero comprar 2 caixas de analgésico". O motor de NLU precisa estruturar a solicitação separando a intenção principal e os parâmetros (entidades).

1. Escreva o trecho de um código em Python que armazene essa frase em um dicionário estruturado contendo as chaves "intencao", "produto" e "quantidade", atribuindo os valores corretos extraídos da frase. Imprima o dicionário.
2. Na arquitetura de NLU, qual a diferença conceitual entre Intenção (Intent) e Entidade?

a) Resposta:

```
frase = "Quero comprar 2 caixas de analgésico"
```

```
# Estruturação em Dicionário
```

```
solicitacao = {  
    "intencao": "comprar_medicamento",  
    "produto": "analgésico",  
    "quantidade": 2  
}
```

```
print(solicitacao)
```

b) Resposta Esperada:

- Intenção (Intent): Representa o objetivo global/ação que o usuário quer realizar (ex: comprar_medicamento ou consultar_saldo).
- Entidade (Entity): São os dados/variáveis específicos que complementam e parametrizam a ação (ex: o nome do remédio analgésico e a quantidade 2).

QUESTÃO 02 – VALOR 0,25 PONTO

Um sistema de busca utiliza Word Embeddings para calcular a proximidade semântica (distância) entre palavras. O sistema calcula a distância entre "aluguel" e "locação" como 0.05 (muito próximas) e entre "aluguel" e "futebol" como 0.95 (muito distantes).

1. Escreva o trecho de uma função chamada `sao_sinonimos(distancia)` que receba o valor numérico da distância entre duas palavras. Se a distância for menor que 0.20, retorne True (são sinônimos). Caso contrário, retorne False. Teste com a distância 0.05.
2. Por que um modelo baseado puramente em TF-IDF não consegue entender sinônimos, enquanto o Word Embeddings consegue?

a) Resposta:

```
def sao_sinonimos(distancia):  
    # Se a distância for menor que 0.30, são sinônimos  
    return distancia < 0.30
```

Teste com a distância de 0.15

```
print(sao_sinonimos(0.15))
```

Saída esperada: True

b) Resposta Esperada:

- **TF-IDF:** Trabalha de forma vetorial esparsa e isolada (*bag-of-words*). Se a palavra "locação" não foi escrita exatamente no texto de treino, o TF-IDF a trata como um termo 100% desconhecido, sem conexão com "aluguel".
- **Word Embeddings:** Representa palavras em vetores densos baseados no **contexto de uso**. Como "aluguel" e "locação" aparecem cercadas de palavras similares no idioma, seus vetores ficam matematicamente próximos no espaço semântico.

MODELO 4

QUESTÃO 01 – VALOR 0,25 PONTO

Um modelo de análise de sentimento para suporte técnico recebe a mensagem: "O sistema está muito lento hoje!". Para higienizar o texto, o pipeline precisa realizar a tokenização e remover as pontuações e stopwords.

Sua tarefa:

1. Crie o trecho de um código em Python que pegue a lista bruta = ["O", "sistema", "está", "muito", "lento", "hoje", "!"], remova o sinal "!" e as stopwords ["O", "está"], gerando uma lista `tokens_limpos`.
2. O que são Stopwords e por que sua remoção é importante na etapa de pré-processamento de texto?

a) Resposta:

```
bruta = ["O", "sistema", "está", "muito", "lento", "hoje", "!"]
```

```
stopwords = ["O", "está"]
```

```
pontuacoes = ["!"]
```

```
tokens_limpos = [w for w in bruta if w not in stopwords and w not in pontuacoes]
```

```
print(tokens_limpos)
```

```
# Saída esperada: ['sistema', 'muito', 'lento', 'hoje']
```

b) Resposta Esperada:

- Stopwords são palavras de alta frequência funcional (artigos, preposições, verbos de ligação) que não possuem valor semântico próprio.
- Remove o "ruído" do texto, reduz a dimensão da matriz de dados, economiza processamento e ajuda os algoritmos a focarem nas palavras estruturais que realmente definem o assunto.

QUESTÃO 02 – VALOR 0,25 PONTO

Um modelo de triagem de chamados em um hospital analisa mensagens de pacientes. O sistema avaliou 100 casos graves reais e obteve os seguintes dados:

- Verdadeiros Positivos (VP) = 90 (casos graves detectados)
 - Falsos Negativos (FN) = 10 (casos graves ignorados)
1. Escreva o trecho de um código em Python que calcule o **Recall (Sensibilidade)** utilizando a fórmula: $Recall = \frac{VP}{VP+FN}$. Imprima o resultado em porcentagem.
 2. O **Recall** avalia o foco em evitar Falsos Positivos ou Falsos Negativos? Por que um Recall alto é fundamental em áreas críticas como saúde ou detecção de fraudes?

a) Resposta:

VP = 90 # Casos graves detectados

FN = 10 # Casos graves ignorados

Recall = VP / (VP + FN)

recall = VP / (VP + FN)

print(f'Recall: {recall * 100:.1f}%')

Saída esperada: 90.0%

b) Resposta Esperada:

- O **Recall** foca em evitar os **Falsos Negativos (FN)**. Em saúde ou fraudes, ignorar um caso verdadeiro (Falso Negativo) pode significar a morte de um paciente sem atendimento rápido ou um prejuízo financeiro irreparável. É preferível ter alarmes falsos (Falsos Positivos) do que deixar um caso grave passar despercebido.

MODELO 5

QUESTÃO 01 – VALOR 0,25 PONTO

Um engenheiro de software precisa processar um volume massivo de textos com extrema velocidade em um servidor com recursos limitados. Ele optou por aplicar **Stemming** em vez de Lematização para reduzir as palavras à sua raiz.

As palavras de entrada são: ["correndo", "corredor", "correu"].

1. Crie o trecho de um código em Python que receba a lista ["correndo", "corredor", "correu"] e simule a ação do *Stemmer* aplicando um corte fixo no sufixo de cada palavra, de modo que todas sejam reduzidas ao radical "corr". Imprima a lista resultante.
2. O que caracteriza o **Stemming** em termos de processamento e por que ele é mais rápido do que a Lematização?

a) Resposta:

```
palavras = ["correndo", "corredor", "correu"]
```

```
# Simulação de corte de sufixo (Stemmer fixo)
```

```
radical = [p[:4] for p in palavras] # Pega as 4 primeiras letras ('corr')
```

```
print(radical)
```

```
# Saída esperada: ['corr', 'corr', 'corr']
```

b) Resposta Esperada:

- O Stemming é uma técnica baseada exclusivamente em regras heurísticas de manipulação de strings (corte numérico de sufixos/prefixos).
- **Por que é mais rápido?** Não precisa consultar léxicos, nem realizar parsing sintático ou tabelas complexas de gramática na memória. É apenas uma operação direta de corte de texto (*string slicing*).

QUESTÃO 02 – VALOR 0,25 PONTO

O motor de NLU de um chatbot de atendimento ao cliente recebeu uma frase muito confusa: "Quero ver aquele negócio lá". O modelo calculou as seguintes probabilidades de intenção:

- **consultar_saldo:** 25% (0.25)
- **fazer_pix:** 30% (0.30)
- **falar_com_atendente:** 15% (0.15)

O limiar de confiança mínimo (**Threshold**) configurado no sistema é de 60% (0.60).

1. Crie o trecho de um código em Python com uma lista de probabilidades [0.25, 0.30, 0.15]. Encontre a maior probabilidade da lista (usando a função `max()`) e, usando uma estrutura `if/else` com o limiar de 0.60, imprima "Intenção Confirmada" se for maior ou igual ao limiar, ou "Acionar Fallback" caso contrário.
2. Diante do resultado obtido, o que o mecanismo de **Fallback** deve responder ao usuário e por que essa resposta é necessária nesse cenário?

a) Resposta:

```
probas = [0.25, 0.30, 0.15]
```

```
threshold = 0.60
```

```
maior_confianca = max(probas)
```

```
if maior_confianca >= threshold:
```

```
    print("Intenção Confirmada")
```

```
else:
```

```
    print("Acionar Fallback")
```

```
# Saída esperada: "Acionar Fallback" (pois 0.30 < 0.60)
```

b) Resposta Esperada:

- O chatbot deve responder uma mensagem de desculpa amigável e pedir esclarecimento (ex: *"Desculpe, não consegui entender o que você precisa. Você poderia reescrever sua mensagem ou escolher uma das opções no menu?"*).
- Evita que o sistema tente adivinhar a intenção com baixa certeza e acabe executando um comando errado ou passando uma informação incorreta ao cliente.